



AI Model Comparison

[Clear](#)

Compare OpenAI, Gemini and Llama side by side.

Your Prompt

Write a JavaScript function that removes duplicate numbers from an array.

73/2000

💡 Tip: Use the exact same prompt for a fair comparison.

➤ Submit



OpenAI

gpt-4.1

OpenAI

Response

Certainly! In JavaScript, you can remove duplicates from an array by leveraging a `Set` (which stores only unique values) or by using `filter`. Here are **two methods**:

Method 1: Using `Set`

```
```javascript
```

```
function removeDuplicates(arr) {
 return [...new Set(arr)];
}
```

// Example usage:

```
console.log(removeDuplicates([1, 2, 2, 3, 4, 4, 5])); // [1, 2, 3, 4, 5]
```
```

Method 2: Using `filter`

```
```javascript
function removeDuplicates(arr) {
 return arr.filter((value, index, self) => self.indexOf(value) === index);
}
```

// Example usage:

```
console.log(removeDuplicates([1, 2, 2, 3, 4, 4, 5])); // [1, 2, 3, 4, 5]
```
```

Both methods will **remove duplicate numbers** from the input array.

Note: They also work with arrays containing other data types, but the question is specific to numbers.

Model: gpt-4.1 | Time: 5.57s



Google Gemini

gemini-3.5-flash

Google Gemini

■ Response

Here is the most modern, efficient, and common way to remove duplicate numbers from an array in JavaScript using a `Set`.

Method 1: Using `Set` (Recommended)

A `Set` is a built-in JavaScript object that only allows unique values. By converting the array into a Set, and then back into an array, duplicates are automatically removed.

```
```javascript
function removeDuplicates(arr) {
 return [...new Set(arr)];
}
```

// Example usage:

```
const numbers = [1, 2, 2, 3, 4, 4, 5, 1, 6];
const uniqueNumbers = removeDuplicates(numbers);

console.log(uniqueNumbers);
// Output: [1, 2, 3, 4, 5, 6]
```
```

Why this is the best approach:

* **Time Complexity:** $O(n)$ (very fast).

* **Readability:** It is concise and uses modern ES6 syntax.

Method 2: Using `.filter()` (For older environments)

If you need to support older environments that don't support `Set` (or just want to see the logic), you can use the `.filter()` method.

```
```javascript
function removeDuplicates(arr) {
 return arr.filter((item, index) => arr.indexOf(item) === index);
}
```

// Example usage:

```
const numbers = [1, 2, 2, 3, 4, 4, 5, 1, 6];
console.log(removeDuplicates(numbers));
// Output: [1, 2, 3, 4, 5, 6]
```
```

Note: This method is slower ($O(n^2)$ complexity) because `indexOf` scans the array on every iteration.

Model: gemini-3.5-flash | Time: 35.22s



Groq

llama-3.3-70b-versatile

Groq

■ Response

```
```\njavascript\nfunction removeDuplicates(lst) {\n  return [...new Set(lst)];\n}\n\n// Example usage:\nlet numbers = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7, 8, 8, 9];\nlet uniqueNumbers = removeDuplicates(numbers);\nconsole.log(uniqueNumbers); // Output: [1, 2, 3, 4, 5, 6, 7, 8, 9]\n```\n
```

Model: llama-3.3-70b-versatile | Time: 0.49s



① Responses may differ in accuracy, tone, detail and instruction-following. Verify important information.